

计算机程序设计基础(2)

--- C++程序设计(6)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020. 3.

问题1

有人把重载的关系运算符成员函数写为:

```
bool operator != (String &s1,String &s2)
{
    if(strcmp(s1.p, s2.p)!=0)
        return true;
    else
        return false;
}
```

能否进一步优化, 提高执行效率呢?

答: 可以改写为:

```
bool operator != (String &s1,String &s2)
{
    return strcmp(s1.p, s2.p)!=0 ;
}
```

问题2

析构函数:

```
String :: ~String()
```

```
{
```

```
    delete []p;
```

```
    cout<<"析构函数被调用!"<<p<<endl;
```

```
}
```

中如果先用delete释放了p所指的字符串动态内存块，然后打印字符串，打印出来的将是乱码：葺葺葺葺葺葺葺葺
(修葺房屋的：葺qì，内码为：0xDDDD)

应该先打印字符串，后释放动态内存：

```
String :: ~String()
```

```
{
```

```
    cout<<"析构函数被调用!"<<p<<endl;
```

```
    delete []p;
```

```
}
```

有程序：

```
#include<iostream>
using namespace std;
void main( )
{
    char a[20], *p=new char[20];
    cout <<"a="<< a <<endl; // 局部字符数组未赋初值
    cout <<"p="<< p <<endl; // 动态内存字符数组未赋初值
    delete []p;
    cout <<"p="<< p <<endl; // 动态内存释放后字符数组
}
```

运行结果为：

a=烫烫烫烫烫烫烫烫烫烫烫
p=屯屯屯屯屯屯屯屯屯屯
p=葺葺葺葺葺葺葺葺葺葺葺葺
请按任意键继续...

烫内码为： 0xCCCC

屯内码为： 0xCDCD

葺内码为： 0xDDDD

问题3 关于构造函数的形参

如果构造函数的形参设为double型，而生成对象时赋的值为int型，结果却没有出错，这是为什么？程序里也没有写它的重载函数。而且如果是构造函数的形参设为int型，而生成对象时赋的值为double型，编译器会警告，而且结果是double数都取整了。这里没有double型的重载函数为什么还能执行该构造函数？

```
class Circle
{
public:
    Circle(int x, int y, int r)
    { CenterX = x; CenterY = y; Rad = r;
      cout << "构造函数被调用" << endl;
    }
    .....
private:
    double CenterX, CenterY, Rad;
};
int main( )
{ Circle C1(3.5, 4.5, 5.5);
}
```

问题3 关于构造函数的形参

答：如果构造函数的形参设为double型，而生成对象时赋的值为int型，结果不会出错。这是因为C/C++的自动类型转换规则是按照精度来衡量的，把一个int型甚至long long超长整型数据转换成double类型，不会损失数据的精度，所以在没有int型形参类型的构造函数的情况下，就自动把int型实参转换成double类型，去调用形参double类型的构造函数。

而反过来，而且如果是构造函数的形参设为int型，而生成对象时赋的初值为double型。虽然把一个double型转换成int类型，会损失数据的精度。但是在没有double型形参类型的构造函数的情况下，只能强制把double型实参取整转换成int类型，去调用形参int类型的构造函数。但编译器会警告：

warning C4244: “参数”：从“double”转换到“int”，可能丢失数据

虽然这里没有double型的构造函数，但编译器在基本数据类型之间尽量进行强制数据转换，甚至你把构造函数的形参设为short或char类型也进行这种强制数据转换，以便能调用构造函数。

问题4 关于this指针的问题

```
#include <iostream>
using namespace std;
class Ball
{
public:
    Ball(double x=0,double y=0,double z=0,double r=0):this->x(x){ }
private:
    double x,y,z,r;
};
int main()
{ return 0;
}
```

请问这段程序中Ball构造函数用this指针赋值时为什么会报错呢？如果不用this指针就没有问题。请问这个问题出在哪儿呢？

答：在ppt3中3.8节讲this指针时强调过“this指针只能在成员函数内使用”，你这样在构造函数的构造序列中使用this指针算是在成员函数体外使用this指针了，所以会报错。构造序列会按照类定义的 double x,y,z,r; 顺序逐个构造对象的成员 x,y,z,r，用:x(x),y(y),z(z),r(r) 即可，不需要加this指针。

问题5 构造函数带缺省值和不带缺省值的问题

在程序设计时我发现，如果在定义构造函数时带缺省值，那么在主函数中就不一定需要赋值（不赋值默认为缺省值），但是如果不带缺省值，在主函数中不赋初值是不可以的，对于对象数组，即使我逐个进行赋值，也是不可以的，这应该如何处理？

如果自己不定义构造函数，使用默认的构造函数，是不是能够在定义时先不去赋值，等后面再赋值呢？

还有一个问题就是对`x(x)`，`Point A(B)`这类赋值的使用，什么情况下可以这样赋值呢？

答：是的，如果在定义构造函数时带缺省值，那么在主函数中定义对象时就不一定需要赋值（不赋值默认为缺省值）。但是如果在定义构造函数时不带缺省值，在主函数中定义对象时必须赋初值。对于对象数组，直接用`A a[5]`；定义数组，然后再循环对每个对象赋值确实不行，因为创建对象数组`a`时调用了5次缺省值的构造函数。但在定义对象数组时用无名对象赋初值：`A_a[5]={A(1,1), A(2,2), A(3,3), A(4,4), A(5,5)}`；不会有问题。用创建5个无名对象代替了调用5次缺省值的构造函数构造出数组`a`。

问题5 构造函数带缺省值和不带缺省值的问题

但是对于创建动态对象数组 `A *a= new A[5];` 因为无法同时赋初值，会调用5次缺省值的构造函数构造出动态数组a，如果没有带缺省值的构造函数，即使再循环逐个进行赋值也不行。所以构造函数必须带缺省值。

如果自己不定义构造函数，使用编译器生成的默认的构造函数，可以在定义对象时先不去赋值（想赋值也不允许），等后面再用类似Set去赋值。

对x(x)这类赋值，只能在构造函数和复制构造函数外的构造序列中使用，其它成员函数也不能这么写。Point A(B)要求B是已定义好的对象，用B复制构造一个新对象A。

问题6 关于浅复制以及析构函数的一些问题

问(1) 在我不定义析构函数时，系统会自动生成一个默认析构函数。我希望知道这个析构函数是否会两次释放浅复制的同一块内存而导致程序卡死，于是写了下面的这个浅复制程序：程序运行完成了，没有卡住，那么是否说明系统默认的析构函数不具备释放内存的功能？那么系统默认的析构函数有什么用呢？

```
class Strings {
public:
    Strings(const char* s) { length = strlen(s); str = new char[length + 1];
        strcpy(str, s); cout << "构造函数被调用" << endl; }
    void Set(const char* s) { length = strlen(s); delete[]str;
        str = new char[length + 1]; strcpy(str, s); }
    void Print( ){ cout << str << endl; cout << "length=" << length << endl;}
private:
    int length;
    char* str;
};
int main( )
{ Strings s1("hello"); Strings s2(s1);
  s1.Print( ); s2.Print( );
  s1.Set("new string");
  s1.Print( ); s2.Print( );
}
```

问题6 关于浅复制以及析构函数的一些问题

答：(1) 在类中如果不定义析构函数，编译系统会自动生成一个默认的析构函数。既然编译系统自动生成的复制构造函数是浅复制，那么相应自动生成的这个析构函数的功能也是“浅析构”，会释放掉对象本身每个成员所占的内存，包括指针成员变量所占的内存，但不会用`delete`去释放指针所指的动态内存块。因此执行两次析构函数不会导致程序出现致命错误。

问(2) 我顺便发现这个浅复制程序的运行结果很奇怪：按道理在重新set字符串s1为“new string”以后，s2也应该跟着变成“new string”了，但是打印出来要么是“葺葺葺葺葺葺葺葺葺”这样一行字，要么是“new string”（对，我运行了几次每次打印出来的还不太一样.....）请问这是为什么？

答：(2) 浅复制会导致两个对象s1和s2的字符串指针str指向同一块动态内存。所以在重新set字符串s1为“new string”以后，因为set函数中有`delete[]str; str = new char[length + 1];`释放并重新申请动态内存，新申请的动态内存有可能与原来的str地址不同。这就造成s2有可能变成“new string”（s1申请的恰好是同一起始地址的动态内存块），s2有可能变成是“葺葺葺葺葺葺葺葺葺”（s1原来的动态内存被`delete`释放了，在另外的地址上申请了新的动态内存）。

问题7 我在上面的程序的基础上又加上了自己定义的析构函数：

//析构函数

```
Strings::~~Strings()
```

```
{ cout << "析构函数被调用！" << endl;  
}
```

结果和没写析构函数时是一样的，这是否说明了自己定义的析构函数并不默认具有释放对象中的内存的能力？只有自己人为地写上"delete []str;"这样一句话，析构函数才能够去释放内存？

答：是的，这样写析构函数除了打印信息，并释放对象成员所占内存外，并不默认具有用delete释放对象中成员指针所指动态内存的能力，必须人为地写上"delete []str;"才行。

问题8 我又把上面程序中的所有的*str换成了str[100], 也就是照着PPT上的程序。

我的理解是这样的, 不知道对不对:

在这个程序中, 没有使用new去申请动态内存了, 因而也就不能够用delete去释放数组str[100]。(我也试了一下, 确实, 析构函数一旦写上delete []str就死掉了)

其次, 这个程序本质上仍是一个浅复制程序, 但是运行结果中显示, 在改变s1后s2仍为hello, 并未随着s1改变, 这只能说明两个对象的数组str[100]分别指向两块不同的内存。浅复制虽不规范, 但仍然勉强能用。

答: 如果类String中的所有的*str换成了str[100], 那么长度为100的str数组是对象成员的一部分, 对象s1和s2的str数组是相互独立的。生成对象时会自动开辟str数组, 浅复制也会复制str数组的内容, 析构函数也会自动释放str数组。不是new生成的数组, 绝不能用delete去释放。

问题9 析构的顺序

```
Constructor! (5,4)    构造对象c1, 初值(5,4)
Constructor! (2,10)   构造对象c2, 初值(2,10)
Constructor! (0,0)    构造对象c3, 缺省初值(0,0)
c1=(5,4)              打印c1值(5,4)
c2=(2,10)             打印c2值(2,10)
Copy Constructor!(2,10) 自右向左传递函数参数, 复制构造c2
Copy Constructor!(5,4)  自右向左传递函数参数, 复制构造c1
Constructor! (0,0)     构造局部对象t, 缺省初值(0,0)
Copy Constructor!(3,-6) 由对象t复制构造一个无名对象返回
Destructor! (3,-6)     析构局部对象t
Destructor! (5,4)       析构参数c1
Destructor! (2,10)      析构参数c2
Destructor! (3,-6)      赋值完成后, 析构sub返回的无名对象
c3=c1-c2=(3,-6)        打印c3值(3,-6)
Copy Constructor!(2,10) 为add传递函数参数, 复制构造c2
Constructor! (7,14)     构造一个无名对象返回
Destructor! (2,10)      析构参数c2
Destructor! (7,14)      赋值完成后, 析构add返回的无名对象
c3=c1+c2=(7,14)        打印c3值(7,14)
Destructor! (7,14)      析构对象c3
Destructor! (2,10)      析构对象c2
Destructor! (5,4)       析构对象c1
请按任意键继续...
```

想请问一下，这个例子的运行结果中，橙色勾出来的部分是因为什么？这四个如果按照构造的先后顺序去析构，应该是“析构无名对象-析构t-析构（5,4）-析构（2,10）”，同样，下面那两个也应该先析构无名对象，再析构参数c2呀？可是为什么运行结果不是这样的呢？

答：当一个函数返回无名对象执行结束，会先析构函数中的局部对象，再析构形参对象（自左至右），等函数返回的无名对象生命周期结束（比如赋值完）后再析构掉这个无名对象。所以在这里不是严格按照“析构顺序与构造顺序完全相反”的原则，所返回的无名对象的析构有时候会延后很久。像s5=s1+s2+s3+s4; 中 s1+s2所返回的无名对象，必须等(s1+s2+s3)+s4所返回的无名对象和(s1+s2)+s3所返回的无名对象析构后才能析构。

补充：常函数与常对象

- 一个类中可以存在重载的同名同参数的成员函数与常成员函数。

例如： `void Print( )` 与 `void Print( ) const`

- 也可以存在重载的同名的成员函数，参数仅仅是常量与非常量（引用与常引用）之分。

例如： `Point(Point &p)`

`Point(const Point &p)`

```

#include <iostream>
using namespace std;
class Point      //Point类的声明
{
public:
    Point(int x=0, int y=0)
    { X=x; Y=y; cout<<"Point构造函数"; Print( ); }
    Point(Point &p)
    { X=p.X; Y=p.Y; cout<<"Point复制构造函数 "; Print( ); }
    Point(const Point &p)
    { X=p.X; Y=p.Y; cout<<"const Point复制构造函数 "; Print( ); }
    ~Point() { cout<<"Point析构函数 "; Print( ); }
    void Print( ) { cout<<"("<<X<<","<<Y<<")"<<endl; }
    void Print( ) const { cout<<"const("<<X<<","<<Y<<")"<<endl; }
    int GetX( ) { return X; }
    int GetY( ) { return Y; }
private:
    int X,Y;
};

void main( ) //主函数
{
    Point p1(1,1);      //建立1个Point类对象
    const Point p2(4,5); //建立1个Point类常对象
    p1.Print( ); p2.Print( );
    Point p3(p1);      //由Point类对象复制构造1个Point类对象
    const Point p4(p2); //由Point类常对象复制构造1个Point类常对象
    const Point p5(p1); //由Point类普通对象复制构造1个Point类常对象
    p3.Print( ); p4.Print( ); p5.Print( );
}

```

运行结果:

Point构造函数(1,1)

Point构造函数(4,5)

(1,1)

const(4,5)

Point复制构造函数 (1,1)

const Point复制构造函数 (4,5)

Point复制构造函数 (1,1)

(1,1)

const(4,5)

const(1,1)

Point析构函数 (1,1)

Point析构函数 (4,5)

Point析构函数 (1,1)

Point析构函数 (4,5)

Point析构函数 (1,1)

请按任意键继续...

普通对象和常对象调用了不同的打印函数。

复制构造对象时,常对象和普通对象调用了不同的复制构造函数。

如果将void Print()打印函数和Point(Point &p)复制构造函数去掉,运行结果将是:

运行结果:

Point构造函数const(1,1)

Point构造函数const(4,5)

const(1,1)

const(4,5)

const Point复制构造函数 const(1,1)

const Point复制构造函数 const(4,5)

const Point复制构造函数 const(1,1)

const(1,1)

const(4,5)

const(1,1)

Point析构函数 const(1,1)

Point析构函数 const(4,5)

Point析构函数 const(1,1)

Point析构函数 const(4,5)

Point析构函数 const(1,1)

请按任意键继续...

普通对象可以调用常打印函数打印。

复制构造对象时，普通对象也可以调用常引用的复制构造函数。

但如果将void Print() const 打印函数和Point(const Point &p)复制构造函数去掉，编译结果是：

```
1>c:\test.cpp(23) : error C2662: "Point::Print": 不能将 "this"指针从  
  "const Point"转换为 "Point &"  
1>      转换丢失限定符  
1>c:\test.cpp(25) : error C2558: class"Point": 没有可用的复制构造  
函数或复制构造函数声明为 "explicit"  
1>c:\test.cpp(27) : error C2662: "Point::Print": 不能将 "this"指针从  
  "const Point"转换为 "Point &"  
1>      转换丢失限定符  
1>c:\test.cpp(27) : error C2662: "Point::Print": 不能将 "this"指针从  
  "const Point"转换为 "Point &"  
1>      转换丢失限定符
```

普通对象可以转换为常对象后进行操作，但常对象不能转换为普通对象进行操作。

提示：以后写类的复制构造函数，形参尽量写成常引用方式：A(const &A)，这样通用性更好。

第6次作业:

见网络学堂第6次作业

2题必做, 1题选做

1. 定义一个字符串类String，其私有成员为一个字符指针str。设计带缺省值的构造函数、复制构造函数、析构函数、设置新值函数Set、打印成员值函数Print。借助此String类，将操作符 + 和 = 重载函数设置为String类的成员函数，实现对操作符 + 和 = 的重载，使之能直接把两个String类类型的字符串进行串接并赋值。请设计各种情况调用此 + 和 = 重载操作符，操作符函数形参可以用引用方式，以提高执行效率。例如：

```
String s1("abcd"), s2("efghij"), s3("lmnop"), s4, s5;
```

```
s4 = s1+s2+s3; s4.Print();
```

```
// 输出结果应该是 abcdefghijlmnop
```

```
s5 = s5+s3+s4; s5.Print();
```

```
// 输出结果应该是 lmnopabcdefghijlmnop
```

2. String类的定义同上题。借助此String类，将操作符 += 重载函数设置为String类的成员函数，实现对操作符 += 的重载，使 s1 += s2; 能直接把一个String类类型的字符串s2串接在另一个String类类型的字符串s1尾上，类似 strcat(s1,s2); 的功能，写 += 操作符重载函数时需考虑各种极端情况。请设计各种情况调用此 += 重载操作符，操作符函数形参可以用引用方式，以提高执行效率。例如：

```
String s1("abcd"), s2("efghij"), s3("lmnop");
```

```
s1 += s2; s1.Print(); // 输出结果应该是 abcdefghij
```

```
s3 += s3 += s2; s3.Print();
```

```
// 输出结果应该是 lmnopdefghijlmnopdefghij
```

3*. 将第1题中的+操作符重载函数设置为String类的友元函数，重写程序。

5. 类的继承与派生

- 继承与派生(Derivation and Inheritance)是C++的重要机制，是面向对象程序设计的重要特征。
- 该机制自动为一个类提供来自另一个类的数据结构和操作。
- 这样可以利用已构造好的类生成新类，充分利用已有资源，进行代码复用（code reused）。
- 在构造新类的过程中保持已有类的特性称为继承。
- 在已有类的基础上新增自己的特性而构造新类的过程称为派生。

目录

- 5.1 概述
- 5.2 访问控制
- 5.3 保护成员
- 5.4 友元与继承
- 5.5 访问权限调整
- 5.6 成员名限定
- 5.7 派生类的构造函数与复制构造函数
- 5.8 带基类内嵌对象成员的派生类
- 5.9 动态束定与虚函数
- 5.10 纯虚函数
- 5.11 虚析构函数
- 5.12 多继承
- 5.13 多继承的二义性
- 5.14 虚基类

5.1 概述

类继承的语法形式:

```
class 标识符2 : 访问控制 标识符1  
{ 新成员和成员函数  
};
```

其中:

标识符1 ----- 基类

标识符2 ----- 派生类

访问控制 ----- 类的继承方式, 有三种:

public: 公有继承

protected: 保护继承

private: 私有继承

新成员和成员函数: 指在继承基类的成员和成员函数后, 派生类新增加的成员和成员函数。

例如: 点类与矩形类

```
class Shape {  
    public: void SetB(int x, int y);  
           void MoveB(int x, int y);  
           void DisplayB( ) const;  
    private: int X, Y;  
};  
class Rectangle: public Shape { // 公有继承  
    public: void SetD(int x, int y, int w, int h);  
           void MoveD(int x, int y );  
           void DisplayD( ) const;  
    private: int Width, Height;  
};
```

- 说明:
- Shape是一个点类， 有：
 - 两个私有成员：X， Y， 表示平面上一个点的坐标（X， Y）；
 - 三个成员函数：SetB(), MoveB(), DisplayB()
- Rectangle是矩形类， 是Shape类的派生类， 有：
 - 四个私有成员：X， Y， Width, Height
 - 六个成员函数：SetB(), MoveB(), DisplayB(), SetD(), MoveD(), DisplayD()
- Rectangle类公有继承了基类Shape类的全部成员和成员函数， 并加入了新的成员和成员函数。

Rectangle:

SetD(), MoveD(), DisplayD()

int Width, Height;

int x,y;

SetB(), MoveB(), DisplayB()

5.2 访问控制（继承性质）

- ◆ 当继承性质为public（公有继承），则派生类全部公有继承基类的成员和成员函数，访问权限保持不变。派生类的成员函数只能访问基类中具有公有访问特性的成员或成员函数。
- ◆ 当继承性质为private（私有继承），则基类的任何性质的成员或成员函数都变成了派生类的私有成员或成员函数，除了派生类的成员函数可以直接访问其基类的公有和保护成员(函数)外，禁止派生类外对任何性质(包括公有)的基类成员或成员函数的直接访问。
- ◆ 通过不同性质的继承，派生类改变了对基类的成员和成员函数的访问权限。

访问权限的继承和调整

继承性质	基类中成员(函数)的访问权限	派生类中成员(函数)的访问权限
public	public protected private	public protected private
protected	public protected private	protected protected private
private	public protected private	private private private

例如5.1节的类中：

```
void Shape::SetB(int x, int y)
{ X=x; Y=y; }
void Rectangle::SetD(int x, int y, int w, int h)
{   SetB(x, y); // 不可以 X=x, Y=y;
    Width = w;
    Height = h;
}
```

虽然Rectangle类继承了Shape类的所有成员和成员函数，Shape类的私有成员X，Y也是Rectangle类的私有成员，但Rectangle类的成员函数SetD()仍不能直接访问X，Y，而要通过基类Shape的公有成员函数SetB()间接访问。

```

#include <iostream>
using namespace std;
class Shape {
    public: void SetB(int x, int y);
           void MoveB(int x, int y);
           void DisplayB( ) const; // 常函数
    private: int X, Y;
};
class Rectangle: public Shape {    // 公有继承
    public: void SetD(int x, int y, int w, int h);
           void MoveD(int x, int y);
           void DisplayD( ) const; // 常函数
    private: int Width, Height;
};
void Shape::SetB(int x, int y)
{ X=x; Y=y; }
void Shape::MoveB(int x, int y)
{ X += x; Y += y; }

```

```

void Shape::DisplayB( ) const
{ cout << X << ' ' << Y << endl; }
void Rectangle::SetD(int x, int y, int w, int h)
{   SetB(x, y);      // 不可以 X=x, Y=y;
    Width = w;   Height = h;
}
void Rectangle::MoveD(int x, int y)
{   MoveB(x, y);    // 不可以 X += x, Y += y;
}
void Rectangle::DisplayD( ) const
{   DisplayB( );    // 不可以 cout << X << ' ' << Y << endl;
    cout << ' ' << Width << ' ' << Height << endl;
}
void main( )
{   Rectangle R;    R.SetD(5, 10, 12, 14);
    Shape S;        S.SetB(2, 4);
    R.DisplayD( );  S.DisplayB( );
}

```

运行结果:

5 10

12 14

2 4

```

#include <iostream> // 私有方式继承的例子
using namespace std;
class A {
public:
    A(int a,int b) : a(a),b(b){ }
    void print(){cout << a << endl; }
    int b; // b为A类的公有成员
private:
    int a; // a为A类的私有成员
};
class B : private A { // B类以私有方式继承A类
public:
    B(int a, int b, int c) : A(a, b),c(c){ }
    void print( )
    { cout << c << endl; //成员函数可直接访问本类的私有成员
      cout << b << endl; //成员函数可直接访问基类的公有成员
      A::print(); //成员函数不可以直接访问基类的私有成员a
      // 若直接写 cout << a << endl; 编译将出现错误:
    } // error C2248: "A::a": 无法访问private 成员(在“A”类中声明)
private:
    int c; // c为B类的私有成员
};

```

```

class C : public B { // C类以公有方式继承B类
public:
    C(int a, int b, int c, int d) : B(a, b,c),d(d){ }
    void print( )
    { cout << d << endl;
      B::print(); // 调用基类B的成员函数间接打印c,b,a
      // 若直接写 cout << c << endl; 编译将出现错误:
      // error C2248: “B::c”: 无法访问private 成员(在“B”类中声明)
      // 若直接写 cout << b << endl; 编译将出现错误:
      // error C2247: “A::b”不可访问, 因为“B”使用“private”从“A”继承
      // 用基类B的基类A的公有成员函数 A::print( ); 打印, 编译将出现错误:
      // error C2247: “A::print”不可访问, 因为 “B”使用 “private”从 “A”继承
    }
private:
    int d; // d为C类的私有成员
};

void main( )
{ C c(1,2,3,4);
  c.print( );
}

```

运行结果:

4
3
2
1

请按任意键继续...

5.3 保护成员

用protected声明的成员被称为保护成员。

```
class MyClass {  
    public:    void Print( );  
              void Set( );  
              int p;  
    protected: int val;  
    private:   int X, Y;  
};  
  
void main( )  
{ MyClass My;  
  My.Set( );  
  My.p=5;  
  My.val = 10; // 不允许直接访问，只有成员函数可以  
  My.Print( );  
}
```

5.3 保护成员(续1)

保护成员具有双重身份：

- 对于建立它的类，它与private成员的性质相似。
- 对于继承此类建立的类，它与public成员性质相似。

```
class Yourclass : public MyClass
{ public: void Update(int x, int y, int v);
};
void Yourclass::Update(int x, int y, int v)
{ Set(x,y,v);
  val = v; // Yourclass类可以直接访问val, 但不能直接访问X,Y
}
void main( )
{ Yourclass You;
  You.Update(3,5,7);
  You.Print( );
  You.val = 10; // 不允许直接访问val, 只有成员函数可以
}
```

5.3 保护成员(续2)

需要注意的是：

在main函数中不能出现 `You.val=9;` 说明val仍不是Yourclass真正的public成员，只是Yourclass的protected成员，算Yourclass类的private成员。

归纳一下：

5.3 保护成员(续3)

若A是基类，B是基于A的public派生类，则：

- A的private成员(函数)，成为B的private成员，但B的成员函数不能直接访问A的私有成员，而只能通过A的成员函数间接访问，它们仍然是A的private成员，但A是B的一部分。
- A的protected成员(函数)，成为B的protected成员(函数)，与B的protected成员(函数)融为一体。B的成员函数可以直接访问基类A的保护成员，但B类外的函数不能直接访问基类A的保护成员。
- A的public成员(函数)，成为B的public成员(函数)，与B的public成员(函数)融和成一体。

```

#include <iostream> // 保护成员继承的例子
using namespace std;
class A {
public:
    A(int a,int b) : a(a), b(b){ }
    void print( ){cout << a << endl; }
    int b; // b为A类的公有成员
protected:
    int a; // a为A类的保护成员
};
class B : public A { // B类以公有方式继承A类
public:
    B(int a, int b, int c) : A(a, b), c(c){ }
    void print( )
    { cout << c << endl; // 成员函数可直接访问本类的保护成员
      cout << b << endl; // 成员函数可直接访问基类的公有成员
      cout << a << endl; // 成员函数可直接访问基类的保护成员
    }
protected:
    int c; // c为B类的保护成员
};

```

```
class C : public B { // C类以公有方式继承B类
public:
```

```
    C(int a, int b, int c, int d) : B(a, b, c), d(d){ }
```

```
    void print( )
```

```
    { cout << d << endl; // 成员函数可直接访问本类的保护成员
```

```
      cout << c << endl; // 成员函数可直接访问基类B的保护成员
```

```
      cout << b << endl; // 成员函数可直接访问基类A的保护成员
```

```
      cout << a << endl; // 成员函数可直接访问基类A的公有成员
```

```
    }
```

```
protected:
```

```
    int d;    // d为C类的保护成员
```

```
};
```

```
void main( )
```

```
{ C c(1,2,3,4);
```

```
  c.print( );
```

```
  c.B::print( );
```

```
  c.A::print( ); // 也可以写为: c.B::A::print( );
```

```
  cout << c.b << endl; //可以直接访问基类的公有成员
```

```
  // cout << c.a << endl; // error C2248: "A::a": 无法访问 protected 成员(在 "A"类中声明)
```

```
  // cout << c.c << endl; // error C2248: "B::c": 无法访问 protected 成员(在 "B"类中声明)
```

```
  // cout << c.d << endl; // error C2248: "C::d": 无法访问 protected 成员(在 "C"类中声明)
```

```
}
```

运行结果:

4

3

2

1

3

2

1

1

2

请按任意键继续...

5.4 友元与继承

由于派生类不能直接访问基类中的私有成员，所以若一个派生类要直接访问基类中的私有成员，可以将这个派生类声明为基类的友元。

```
class Myclass {
{   friend class Yourclass;
    public: void Set(int x, int y);
    private: int X, Y;
}
class Yourclass : public Myclass {
    public: void Set(int x, int y, int val);
           void Print( );
    private: int val;
}
```

5.4 友元与继承(续1)

```
void Myclass::Set(int x, int y)
{ X=x; Y=y; }
void Yourclass::Set(int x, int y, int v)
{ X=x; Y=y; // 可以直接访问基类的私有成员X,Y
  // Myclass::Set(x, y); //也可以调用基类的公有函数访问x,y
  val=v;
}
void Yourclass::Print( )
{ cout <<X <<' ' <<Y <<' ' <<val <<endl;}
  // 可以直接访问并打印基类的私有成员X,Y
void main( )
{ Yourclass Obj;
  Obj.Set(1, 3, 5);
  Obj.Print( );
}
```

5.5 访问权限调整

- ◆ 在私有继承的情况下，基类的所有成员和成员函数在派生类中具有私有成员的特性。
- ◆ 通过访问权限调整，可恢复派生类所继承的成员在基类中的访问权限。

```
class Myclass {  
    public: int f( );  
        void f(int i);  
        void g( );  
    protected: void h( );  
    private: int X;  
};
```

5.5 访问权限调整(续1)

```
class Yourclass : private Myclass {  
    public: void k( );  
        void hh( );  
        Myclass::f; // 恢复f的公有属性  
        Myclass::g; // 恢复g的公有属性  
    protected: Myclass::h; // 恢复h的保护属性  
};
```

注意： 只能恢复原状，不能把private, protected成员的访问权限调整为 public 或 protected。

```
#include <iostream>
using namespace std;
class Myclass {
    public: int f( );
           void f(int i);
           void g( );
    protected: void h( );
    private: int X;
};
class Yourclass : private Myclass {
    public: void k( );
           void hh( );
           Myclass::f; // 恢复f的公有属性
           Myclass::g; // 恢复g的公有属性
    protected: Myclass::h; // 恢复h的保护属性
};
int Myclass::f( )
{ return X; }
```

```

void Myclass::f(int i)
{ X=i; }
void Myclass::g( )
{ cout <<X <<endl; }
void Myclass::h( )
{ X++; }
void Yourclass::k( )
{ f( f( ) - 1);} // 成员函数k不能直接访问X，只能调用基类的函数间接访问
void Yourclass::hh( )
{ h( ); } // h是保护函数，外界不能直接访问，只能由本类的成员函数访问
void main( )
{ Yourclass b;
  b.f(5); b.g( ); // 初值置为 5
  b.hh( ); b.g( ); // 加1 写 b.h( );是不可以的
  b.k( ); b.g( ); // 减1
}
运行结果:

```

```

5
6
5
请按任意键继续...

```

5.6 成员名限定

在派生类和基类中可以声明同名的成员(函数)，不特别说明，在派生类中访问时，编译器会默认为是在派生类中的（后出现的），除非用：

基类名::基类成员名

进行成员名限定，才能调用同名的基类成员(函数)

```
class MyClass {  
    public: MyClass(int x);  
           void Print( );  
    private: int X;  
};
```

5.6 成员名限定(续1)

```
class Yourclass : public Myclass {  
    public: Yourclass(int x, int y);  
           void Print( ); //派生类的同名成员函数  
    private: int X;    //派生类的同名私有成员  
};  
  
void Myclass::Print( ) // 基类的Print函数  
{ cout << X << " base class\n";}  
  
void Yourclass::Print( ) // 派生类的Print函数  
{ Myclass::Print( ); // 派生类调用基类的Print必须加前缀  
  // Myclass::, 否则成为自递归函数调用, 会出现运行错误  
  cout << X << " derivation class\n";  
}
```

5.6 成员名限定(续2)

```
Myclass::Myclass(int x)
```

```
{ X=x;}
```

```
Yourclass::Yourclass(int x, int y) : Myclass(x)
```

```
{ X=y;} // 注意：这里不能直接函数调用Myclass(x),
```

```
// 构造函数不能在程序段{ }中被显式调用
```

```
void main( )
```

```
{ Yourclass r(3,5);
```

```
    r.Print( );           // 调用派生类的Print函数
```

```
    r.Myclass::Print( ); // 调用基类的Print函数
```

```
}
```

5.2节例子的另外一种写法（基类函数与派生类函数同名）：

```
#include <iostream>
using namespace std;
class Shape {
    public: void Set(int x, int y);
           void Move(int x, int y);
           void Display( ) const;
    private: int X, Y;
};
class Rectangle: public Shape {
    public: void Set(int x, int y, int w, int h);
           void Move(int x, int y);
           void Display( ) const;
    private: int Width, Height;
};
void Shape::Set(int x, int y)
{ X=x; Y=y; }
void Shape::Move(int x, int y)
{ X += x; Y += y; }
```

```

void Shape::Display( ) const
{ cout << X << ' ' << Y << endl; }
void Rectangle::Set(int x, int y, int w, int h)
{   Shape :: Set(x, y); // 这里必须加限定名Shape ::
    Width = w;   Height = h;
}
void Rectangle:: Move(int x, int y)
{   Shape :: Move(x, y); // 这里必须加限定名Shape ::
}
void Rectangle::Display( ) const
{   Shape :: Display( ); // 这里必须加限定名Shape ::
    cout << ' ' << Width << ' ' << Height << endl;
}
void main( )
{   Rectangle R; R.Set(5,10,12,14); R.Display( );
    R.Move(+1,-1); R.Display( );
    Shape S;   S.Set(2, 4);   S.Display( );
}

```

运行结果:

5 10

12 14

6 9

12 14

2 4

如果Rectangle::Set()写为:

```
void Rectangle::Set(int x, int y, int w, int h)
{
    Set(x, y); // 这里不加以限定名Shape ::
    Width = w; Height = h;
}
```

编译结果:

Rectangle2.cpp(22): error C2660: “Rectangle::Set”: 函数不接受2 个参数

如果Rectangle:: Display() const 写为:

```
void Rectangle::Display( ) const
{
    Display( ); // 这里不加以限定名Shape ::
    cout << ' ' << Width << ' ' << Height << endl;
}
```

编译结果:

Rectangle2.cpp(31) : warning C4717: “Rectangle::Display”: 如递归所有控件路径, 函数将导致运行时堆栈溢出

运行结果:



这里： `void Shape::Display( ) const`
和 `void Rectangle::Display( ) const`
不是重载函数，因为两个函数一模一样，而且不是同一个类的。

同样： `void Shape::Set(int x, int y)`
和 `void Rectangle::Set(int x, int y, int w, int h)`
也不是重载函数，因为他们不是同一个类的。

注意：

派生类的同名函数掩蔽了基类的同名函数，除非加限定名 **Shape ::** 指明要使用基类的同名函数，否则在编译器看来是直接自递归调用派生类的函数本身，造成要么编译错误，要么运行错误（无条件直接自递归调用）。

5.7 派生类的构造函数与复制构造函数

- ◆ 派生类的构造函数与复制构造函数如何写？

若B是基类，A是派生类，

- ◆ 则派生类A的构造函数的书写格式为：

```
A::A(参数表): B(参数) // 先构造基类无名对象B
{ .....
}
```

- ◆ 派生类A的复制构造函数的书写格式为：

```
A::A(const A &p): B(p) // 注意：这里B的参数为A类对象p,
// 用对象p中基类部分去复制构造无名基类对象B
{ .....
}
```

```

#include <iostream>
using namespace std;
class Shape {
    public: Shape(int x=0, int y=0);
           Shape(const Shape &p);
           ~Shape( );
           void Set(int x, int y);
           void Move(int x, int y);
           void Display( ) const;
    private: int x, y;
};
Shape::Shape(int x, int y) : x(x), y(y)
{   cout << "Shape constructor " ; Display( ); }
Shape::Shape(const Shape &p) : x(p.x), y(p.y)
{   cout << "Shape copy constructor "; Display( ); }
Shape::~~Shape( )
{   cout << "Shape Destructor " ; Display( ); }

```

```

void Shape::Set(int x, int y)
{ this->x=x; this->y=y; }
void Shape::Move(int x, int y)
{ this->x += x; this->y += y; }
void Shape::Display( ) const
{ cout << x << ',' << y << endl; }
class Rectangle: public Shape {
    public: Rectangle(int x=0, int y=0, int w=0, int h=0);
           Rectangle(const Rectangle &p);
           ~Rectangle( );
           void Set(int x, int y, int w, int h);
           void Move(int x, int y);
           void Display( ) const;
    private: int Width, Height;
};

```

```

Rectangle::Rectangle(int x, int y, int w, int h) : Shape(x,y)
{ Width=w; Height=h;
  cout << "Rectangle constructor "; Display( );
}
Rectangle::Rectangle(const Rectangle &p) : Shape(p)
{ Width=p.Width; Height =p. Height;
  cout << "Rectangle copy constructor "; Display( );
}
Rectangle::~Rectangle( )
{ cout << "Rectangle Destructor "; Display( );
}
void Rectangle::Set(int x, int y, int w, int h)
{ Shape::Set(x, y); // 不可以 this->x = x;  this->y = y;
  Width = w;  Height = h;
}

```

```

void Rectangle::Move(int x, int y)
{   Shape::Move(x, y); // 不可以 this->x += x; this->y += y;
}

void Rectangle::Display( ) const
{   Shape::Display( ); // 不可以 cout <<x<<','<<y<<endl;
    cout << ' ' << Width << ' ' << Height << endl;
}

void main( )
{   Shape S;          S.Display( );
    S.Set(2, 4);      S.Display( );
    S.Move(1, 1);     S.Display( );
    Rectangle R(1, 1, 10, 10); R.Display( );
    R.Set(5, 10, 12, 14); R.Display( );
    R.Move(-1, -2);   R.Display( );
    Rectangle T(R);   T.Display( );
    T.Move(+4, +4);   T.Display( );
}

```

运行结果:

Shape constructor 0,0

0,0

2,4

3,5

Shape constructor 1,1

Rectangle constructor 1,1

10 10

1,1

10 10

5,10

12 14

4,8

12 14

Shape copy constructor 4,8

Rectangle copy constructor 4,8

12 14

4,8

12 14

8,12

12 14

Rectangle Destructor 8,12

12 14

Shape Destructor 8,12

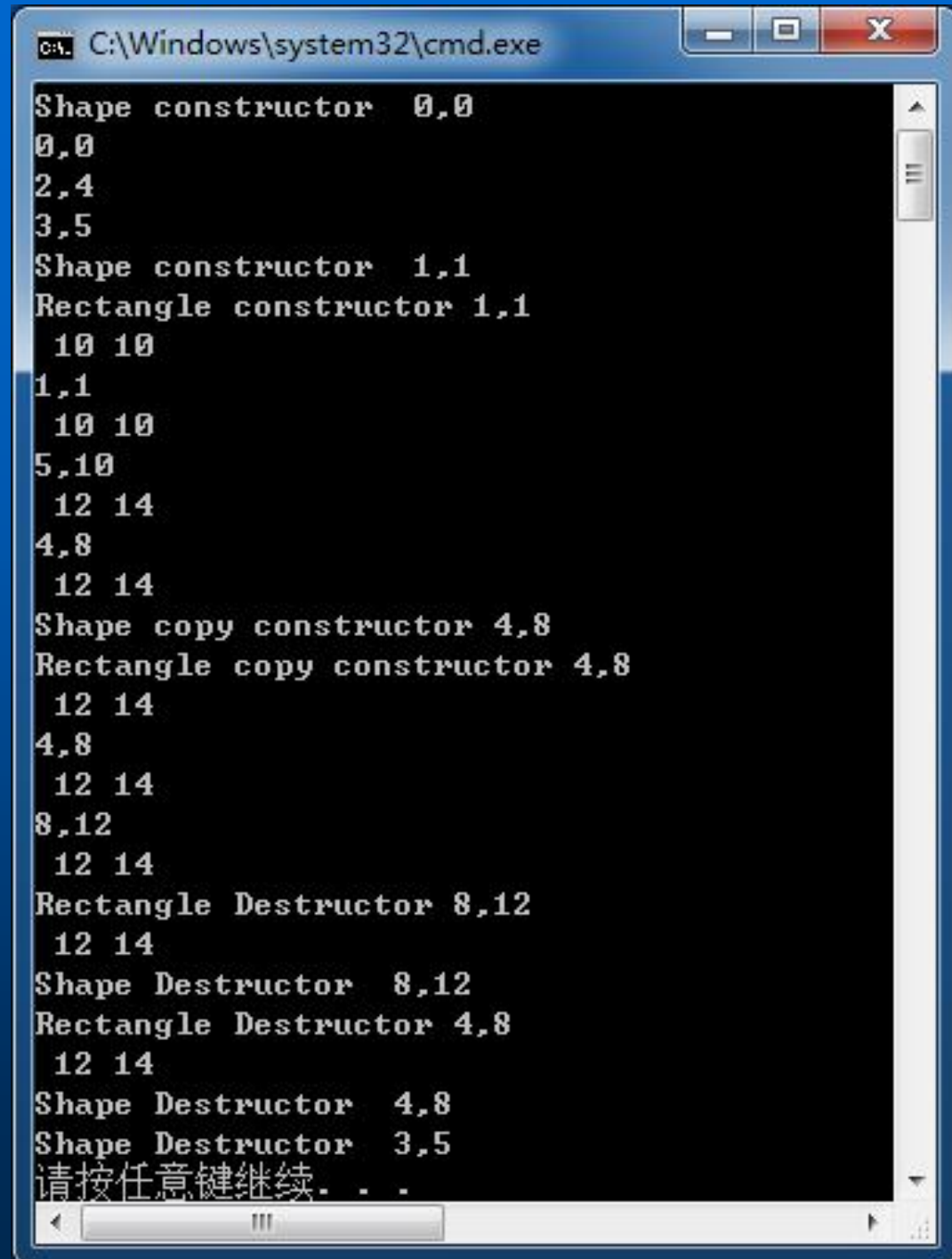
Rectangle Destructor 4,8

12 14

Shape Destructor 4,8

Shape Destructor 3,5

请按任意键继续...



```
C:\Windows\system32\cmd.exe

Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 1,1
Rectangle constructor 1,1
10 10
1,1
10 10
5,10
12 14
4,8
12 14
Shape copy constructor 4,8
Rectangle copy constructor 4,8
12 14
4,8
12 14
8,12
12 14
Rectangle Destructor 8,12
12 14
Shape Destructor 8,12
Rectangle Destructor 4,8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...
```

如果把派生类Rectangle的构造函数、复制构造函数改写为:

```
Rectangle::Rectangle(int x, int y, int w, int h) //: Shape(x,y)
{ Width=w; Height=h;
  cout << "Rectangle constructor ";
  Shape::Display( );
  cout <<Width << ','<<Height<<endl;
}
```

```
Rectangle::Rectangle(const Rectangle &p) // : Shape(p)
{ Width=p.Width; Height =p. Height;
  cout << "Rectangle copy constructor ";
  Shape::Display( );
  cout <<Width << ','<<Height<<endl;
}
```

不调用基类的(复制)构造函数, 运行结果会是什么?

运行结果:

Shape constructor 0,0

0,0

2,4

3,5

Shape constructor 0,0

Rectangle constructor 0,0

10 10

0,0

10 10

5,10

12 14

4,8

12 14

Shape constructor 0,0

Rectangle copy constructor 0,0

12 14

0,0

12 14

4,4

12 14

Rectangle Destructor 4,4

12 14

Shape Destructor 4,4

Rectangle Destructor 4,8

12 14

Shape Destructor 4,8

Shape Destructor 3,5

请按任意键继续...

Rectangle的构造函数、复制构造函数通过调用缺省参数的Shape构造函数构造了基类Shape，但Rectangle的构造函数没有给基类的Shape赋值1,1。而Rectangle的复制构造函数也没有真正复制p的Shape值给新的对象。

派生类Rectangle的构造函数、复制构造函数还可以写为：

```
Rectangle::Rectangle(int x, int y, int w, int h)
```

```
    : Shape(x,y) , Width(w), Height(h)
```

```
{ cout << "Rectangle constructor ";   Display( );  
}
```

```
Rectangle::Rectangle(const Rectangle &p) : Shape(p) ,
```

```
    Width(p.Width), Height(p.Height)
```

```
{ cout << "Rectangle copy constructor ";   Display( );  
}
```

从理论上讲，这样书写运行效率更高一些。是在构造Width 和 Height时，同时赋初值。

注意： **Shape(p)** 不要写为： **Shape(&p)**

派生类Rectangle的构造函数、复制构造函数不能写为:

```
Rectangle::Rectangle(int x, int y, int w, int h)
```

```
{ Shape(x,y);  
  Width=w; Height=h;  
  cout << "Rectangle constructor ";  Display( );  
}
```

```
Rectangle::Rectangle(const Rectangle &p)
```

```
{ Shape(p);  
  Width=p.Width; Height =p. Height;  
  cout << "Rectangle copy constructor ";  Display( );  
}
```

这样Rectangle复制构造函数编译会指示Shape(p); 行有错误:

error C2082: 形参“p”的重定义

改为强制类型转换:

```
Rectangle::Rectangle(const Rectangle &p)
```

```
{ Shape((Shape&)p);  
  Width=p.Width; Height =p. Height;  
  cout << "Rectangle copy constructor ";  Display( );  
}
```

运行结果:

```
Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 0,0
Shape constructor 1,1
Shape Destructor 1,1
Rectangle constructor 0,0
10 10
0,0
10 10
5,10
12 14
4,8
12 14
Shape constructor 0,0
Shape copy constructor 4,8
Shape Destructor 4,8
Rectangle copy constructor 0,0
12 14
0,0
12 14
4,4
12 14
Rectangle Destructor 4,4
12 14
Shape Destructor 4,4
Rectangle Destructor 4,8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...
```

原来正确的运行结果:

```
Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 1,1
Rectangle constructor 1,1
10 10
1,1
10 10
5,10
12 14
4,8
12 14
Shape copy constructor 4,8
Rectangle copy constructor 4,8
12 14
4,8
12 14
8,12
12 14
Rectangle Destructor 8,12
12 14
Shape Destructor 8,12
Rectangle Destructor 4,8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...
```

分析：

1. Rectangle的构造函数、复制构造函数通过调用缺省值的Shape构造函数构造了Rectangle对象中的Shape基类部分，但Rectangle的构造函数没有给Rectangle对象中的Shape基类赋值(1,1)，而Rectangle的复制构造函数也没有真正复制p的Shape值(4,8)给新的Rectangle对象中的Shape基类。
2. Rectangle构造函数中的Shape(x,y); 实际上是构造了一个初值为(1,1) 的临时Shape无名对象，而且构造完成后，因为没什么用处，立刻就被析构掉了。
3. Rectangle复制构造函数中的Shape((Shape&)p); 实际上是复制构造了一个初值为(4,8) 的临时Shape无名对象，而且构造完成后，因为没什么用处，立刻也被析构掉了。

上例的基类private成员如果改为protected成员：

把私有成员改为保护成员后的例子：

```
#include <iostream>
using namespace std;
class Shape {
    public: Shape(int x=0, int y=0);
           Shape(const Shape &p);
           ~Shape( );
           void Set(int x, int y);
           void Move(int x, int y);
           void Display( ) const;
    protected: int x, y;
};
Shape::Shape(int x, int y)
{ this->x = x;  this->y = y;
  cout << "Shape constructor "; Display( );
}
Shape::Shape(const Shape &p)
{ this->x=p.x;  this->y=p.y;
  cout << "Shape copy constructor "; Display( );
}
```

```

Shape::~~Shape( )
{   cout << "Shape Destructor  "; Display( ); }
void Shape::Set(int x, int y)
{ this->x=x;  this->y=y; }
void Shape::Move(int x, int y)
{ this->x += x;  this->y += y; }
void Shape::Display( ) const
{ cout << x << ',' << y << endl; }
class Rectangle: public Shape {
    public: Rectangle(int x=0, int y=0, int w=0, int h=0);
           Rectangle(const Rectangle &p);
           ~Rectangle( );
           void Set(int x, int y, int w, int h);
           void Move(int x, int y);
           void Display( ) const;
    protected: int Width, Height;
};

```

```

Rectangle::Rectangle(int x, int y, int w, int h) : Shape(x,y),
    Width(w), Height(h)
{ cout << "Rectangle constructor ";
  Display( );
}
Rectangle::Rectangle(const Rectangle &p) : Shape(p),
    Width(p.Width), Height(p.Height)
{ cout << "Rectangle copy constructor ";
  Display( );
}
Rectangle::~Rectangle( )
{ cout << "Rectangle Destructor "; Display( );
}
void Rectangle::Set(int x, int y, int w, int h)
{  this->x = x;  this->y = y; // 可以直接访问基类保护成员
   Width = w;   Height = h;
}

```

```

void Rectangle::Move(int x, int y)
{
    this->x += x;  this->y += y; // 可以直接访问基类保护成员
}
void Rectangle::Display( ) const
{
    cout<<' ' <<x <<' ' <<y <<endl // 可以直接输出基类保护成员
        <<' ' << Width <<' ' << Height << endl;
}
void main( )
{
    Shape S;          S.Display( );
    S.Set(2, 4);      S.Display( );
    S.Move(1, 1);     S.Display( );
    Rectangle R(1, 1, 10, 10); R.Display( );
    R.Set(5, 10, 12, 14); R.Display( );
    R.Move(-1, -2);   R.Display( );
    Rectangle T(R);   T.Display( );
    T.Shape::Move(+4, +4); T.Display( );
}

```

运行结果:

Shape constructor 0,0

0,0

2,4

3,5

Shape constructor 1,1

Rectangle constructor 1 1

10 10

1 1

10 10

5 10

12 14

4 8

12 14

Shape copy constructor 4,8

Rectangle copy constructor 4 8

12 14

4 8

12 14

8 12

12 14

Rectangle Destructor 8 12

12 14

Shape Destructor 8,12

Rectangle Destructor 4 8

12 14

Shape Destructor 4,8

Shape Destructor 3,5

请按任意键继续...

说明:

本例中Rectangle的复制构造函数: //调用基类复制构造函数

```
Rectangle::Rectangle(const Rectangle &p) : Shape(p),  
    Width(p.Width), Height(p.Height)  
{ cout << "Rectangle copy constructor ";  
  Display( );  
}
```

也可以写成（调用基类Shape的构造函数）：

```
Rectangle::Rectangle(const Rectangle &p) : Shape(p.x, p.y),  
    Width(p.Width), Height(p.Height)  
{ cout << "Rectangle copy constructor ";  
  Display( );  
}
```

因为Rectangle的成员函数可以直接访问基类的保护成员。

但不能写成:

```
Rectangle::Rectangle(const Rectangle &p) : x(p.x), y(p.y),  
    Width(p.Width), Height(p.Height)  
{ cout << "Rectangle copy constructor ";  
  Display( );  
}
```

因为x和y是基类成员，只能由基类的构造函数或复制构造函数来构造。 写成:

```
Rectangle::Rectangle(const Rectangle &p)  
{ x= p.x; y=p.y;  
  Width=p.Width; Height=p.Height;  
  cout << "Rectangle copy constructor ";  
  Display( );  
}
```

马马虎虎还可以，是隐含先用缺省值Shape构造函数构造Shape，(如果Shape构造函数没有缺省值，编译时将出错)再构造Width和Height后，再逐个赋值(两次赋值影响效率)。

```

#include <iostream> //引用派生类对象中的基类部分
using namespace std;
class Base
{ public:
    Base(int x=0): ValB(x) { cout << ValB; }
    ~Base() { cout << ValB; }
    void print( ) { cout << ValB<<endl; }
private:
    int ValB;
};
class Derived : public Base
{ public:
    Derived(int x=0,int y=0): Base(x), ValD(y) { cout << ValD;}
    ~Derived( ) { cout << ValD; }
    void print( ) { Base::print( ); cout << ValD<<endl; }
private:
    int ValD;
};
void main( )
{Derived Obj1(2,3), Obj2(3,4);
  Base &x=Obj1; // 基类引用可以直接引用派生类中继承的相应基类部分
  x.print( ); // =在这里是地址绑定,让x指向Obj1中的Base基类部分的首地址
  Obj1.print( );
}

```

运行结果:

23342

2

3

4332请按任意键继续...

5.8 带基类内嵌对象成员的派生类

若有类Point，私有成员为(X,Y)，若要以Point作为基类，生成派生类Line，Line中希望有两个Point对象(用两个点来表示平面坐标上的一条直线)，若想派生两次Point是不允许的，写为：

```
class Line: public Point, public Point
```

编译结果是：

```
error C2500: “Line”: “Point”已是直接基类
```

因此只能派生一次Point作为基类，另设一个Point内嵌对象作为Line的私有成员：

```
class Line: public Point {  
    public: .....  
    private: Point p2;  
}
```

那么如何写Line的构造函数、复制构造函数及成员函数呢？

```

#include <iostream>
using namespace std;
class Point {
    public: Point(int x=0, int y=0);
           Point(const Point &p);
           ~Point( );
           void Display( ) const;
    private: int X, Y;
};
Point::Point(int x, int y) : X(x), Y(y)
{   cout << "Point constructor " ; Display( ); }
Point::Point(const Point &p) : X(p.X), Y(p.Y)
{   cout << "Point copy constructor "; Display( ); }
Point::~~Point( )
{   cout << "Point Destructor "; Display( ); }
void Point ::Display( ) const
{ cout << X << ',' << Y << endl; }

```

```

class Line: public Point {
    public: Line(int x1=0, int y1=0, int x2=0, int y2=0);
           Line(Point &p1, Point &p2);
           Line(const Line &p);
           ~Line( );
           void Display( ) const;
    private: Point p2;
};

Line::Line(int x1, int y1, int x2, int y2) : Point(x1,y1), p2(x2,y2)
{ cout << "Line constructor "; Display( ); }
Line::Line(Point &p1, Point &p2) : Point(p1), p2(p2)
{ cout << "Line constructor2 "; Display( ); }
Line::Line(const Line &p) : Point(p), p2(p.p2)
{ cout << "Line copy constructor "; Display( ); }
Line::~~Line( )
{ cout << "Line Destructor "; Display( ); }
void Line ::Display( ) const
{ Point::Display( ); p2.Display( ); }

```

```

void main( )
{
    Point p1(1, 2), p2(3, 4); // 构造两个点
    p1.Display( ); p2.Display( );
    Line L1(-1,-1,5,6); // 用两对坐标构造一条线
    L1.Display( );
    Line L2(p1, p2); // 用两个点构造一条线
    L2.Display( );
    Line L3(L2); // 用一条线复制构造另一条线
    L3.Display( );
}

```

注意派生类Line的构造函数、复制构造函数的写法：

Line::Line(int x1, int y1, int x2, int y2) : **Point(x1,y1)**, p2(x2,y2)

Line::Line(Point &p1, Point &p2) : **Point(p1)**, p2(p2)

Line::Line(const Line &p) : **Point(p)**, p2(p.p2)

先直接用Line对象p构造当前对象的基类无名对象Point，再用Line对象p的内嵌对象p.p2构造当前对象的内嵌对象p2。

而Display函数是先调用基类的Display，再调用内嵌对象p2的Display分别显示Line中两个Point对象的坐标。

运行结果为:

```
Point constructor 1,2 // 构造p1
Point constructor 3,4 // 构造p2
1,2
3,4
Point constructor -1,-1 // 构造基类
Point constructor 5,6 // 构造p2
Line constructor -1,-1 // 构造L1
5,6
-1,-1
5,6
Point copy constructor 1,2 // 复制构造基类
Point copy constructor 3,4 // 复制构造p2
Line constructor2 1,2 // 构造L2
3,4
1,2
3,4
Point copy constructor 1,2 // 复制构造基类
Point copy constructor 3,4 // 复制构造p2
Line copy constructor 1,2 // 复制构造L3
```

```
3,4
1,2
3,4
Line Destructor 1,2 // 析构L3
3,4
Point Destructor 3,4 // 析构p2
Point Destructor 1,2 // 析构基类
Line Destructor 1,2 // 析构L2
3,4
Point Destructor 3,4 // 析构p2
Point Destructor 1,2 // 析构基类
Line Destructor -1,-1 // 析构L1
5,6
Point Destructor 5,6 // 析构p2
Point Destructor -1,-1 // 析构基类
Point Destructor 3,4 // 析构p2
Point Destructor 1,2 // 析构p1
请按任意键继续...
```

第7次作业:

见网络学堂第7次作业

前2题必做, 第3题选做